

UNIVERSIDAD MARIANO GÁLVEZ DE GUATEMALA

Análisis de Sistemas II

PROYECTO 1

SISTEMA DE GESTIÓN DE MICROCRÉDITO

Crédito Vecino, S. A.

Curso	Análisis de Sistemas II (037)
Integrantes	Erwin Ricardo Lémus Navarro - 7690-22-4381 Edgar Esaú Contreras García - 7690-23-6131 Edwin Geovany Valle Benito - 7690-19-482 Fredy Roberto Orozco Alvarado - 7690-22-16980
Docente	Ing. Ezequiel Urizar Araujo
Sección	A
Repositorio	https://github.com/egvalle/microcredito-p1.git
Fecha de entrega	28/08/2026

Versión 1.0

Control del documento

Elemento	Valor	Estado
Documento	Proyecto 1 - E1 a E5	Integrado
Versión	1.0	Para revisión
Responsable	Por completar	Pendiente

Propósito y alcance

Este documento consolida el modelado UML y la trazabilidad (E1), la decisión de arquitectura (E2), el diseño de componentes (E3), el núcleo TypeScript y sus pruebas (E4), y los contratos OpenAPI con decisiones arquitectónicas (E5). Los artefactos editables y el código fuente se mantienen en el repositorio del proyecto.

Estado de los entregables

Entregable	Contenido	Estado
E1	UML y trazabilidad	Integrado
E2	Arquitectura, 4+1 y C4	Integrado
E3	Componentes, SOLID/GRASP y patrones	Integrado
E4	Núcleo TypeScript y pruebas	Validado: 58/58
E5	OpenAPI y ADR	En revisión final

Tabla de contenidos

Control del documento	2
Propósito y alcance	2
Estado de los entregables.....	2
Tabla de contenidos.....	3
Alcance del Proyecto	5
E1 - Modelado UML y trazabilidad.....	5
Actores principales	5
Casos de uso	5
Modelo de clases del dominio	5
Secuencia: registrar pago de cuota	6
Secuencia: aprobar y desembolsar crédito.....	6
Estados del crédito	6
Actividad: origen del crédito	6
Matriz de trazabilidad	7
E2 - Decisión y justificación de arquitectura	8
Arquitectura seleccionada.....	8
Atributos de calidad prioritarios	8
Organización modular	9
Arquitectura hexagonal	9
Modelo 4+1.....	9
Vista lógica	9
Vista de desarrollo	9
Vista de escenarios	9
Modelo C4	10
Nivel 1 - Contexto	10
Nivel 2 - Contenedores	10
Nivel 3 - Componentes	10
Justificación arquitectónica	10
E3 - Diseño de componentes, principios y patrones	11
Descomposición de componentes	11
Diseño detallado del cálculo financiero.....	11
Aplicación de SOLID y GRASP	11
Patrones de diseño	12

Cohesión y acoplamiento	12
E4 - Núcleo TypeScript ejecutable y pruebas.....	13
Estructura implementada	13
Representación y redondeo	13
Evidencia de ejecución.....	13
Demostración de cálculos	14
E5 - Contratos de API y decisiones de arquitectura.....	15
Recursos y operaciones.....	15
Convención de errores e idempotencia	15
ADR-001 - Arquitectura hexagonal con monolito modular.....	15
ADR-002 - Representación de importes monetarios	16
Conclusiones y verificación final	17
Glosario	17
Control de cambios	17

Alcance del Proyecto

En esta etapa se integran el análisis, el modelado, el diseño arquitectónico y de componentes, el núcleo financiero ejecutable en TypeScript, sus pruebas y los contratos de API. No se incluye una aplicación web completa, base de datos operativa, autenticación, RAG, chat ni servidor MCP funcional.

E1 - Modelado UML y trazabilidad

El modelado UML representa los principales actores, procesos, entidades y reglas del dominio. Los diagramas se mantendrán en formato PlantUML editable y se exportarán como imágenes para incorporarlos al documento consolidado.

Actores principales

Actor	Responsabilidad
Cliente	Solicita créditos, consulta su crédito y realiza pagos por los canales disponibles.
Asesor de crédito	Registra clientes, solicitudes y realiza la evaluación inicial.
Comité de crédito	Aprueba, rechaza y autoriza determinadas decisiones de crédito.
Empleado de cobros	Registra pagos recibidos por los canales internos.
Canal de pago externo	Representa futuras transferencias, cajeros, pasarelas o integraciones bancarias.
Analista / encargado de cierres	Genera cierres y analiza la cartera.
Gerencia	Consulta información consolidada y cartera en riesgo.

Casos de uso

Los casos principales son: registrar y consultar clientes; solicitar, evaluar, aprobar o rechazar créditos; desembolsar créditos y generar su plan de amortización; registrar pagos; aplicar la prelación de pagos; calcular mora y tramo de atraso; consultar crédito y saldo; generar cierres; consultar cartera en riesgo; reestructurar créditos y declarar créditos incobrables.

Registrar pago se diseña de forma independiente del canal. Actualmente puede ser iniciado por un empleado de cobros y, en el futuro, por un canal externo. La regla financiera aplicada será la misma sin importar el origen.

Modelo de clases del dominio

El modelo incluye Cliente, SolicitudCredito, Credito, PlanAmortizacion, Cuota, Pago, Movimiento, Cierre y el objeto de valor Dinero. Se incorporan servicios de dominio como CalculadoraMora, PrelacionPago y CalculadoraCarteraRiesgo. Como refinamiento del modelo se consideran también PoliticaCredito e HistorialEstadoCredito para representar políticas versionadas y la auditoría de cambios de estado.

Dinero se modela como un objeto de valor inmutable y evita el uso de punto flotante para importes monetarios. El tramo de mora se considera una clasificación derivada de los días de atraso y no un estado independiente.

Secuencia: registrar pago de cuota

El flujo valida primero la idempotencia del pago, recupera el crédito, determina los importes pendientes y calcula la mora. Posteriormente aplica la prelación obligatoria: gastos o comisiones, interés moratorio, interés corriente y capital. Finalmente registra el pago y sus movimientos y recalcula la situación del crédito. Si todo lo vencido queda cubierto, un crédito EN_MORA puede volver a VIGENTE; si el pago es parcial, puede permanecer en mora y cambiar de tramo.

Secuencia: aprobar y desembolsar crédito

El Comité de crédito aprueba únicamente una solicitud válida que cumpla la política aplicable. Para el desembolso se recupera la política financiera vigente, se genera el plan de amortización, se crea el crédito, se registra el movimiento de desembolso y finalmente el crédito pasa por DESEMBOLSADO hasta quedar VIGENTE.

El plan utiliza el sistema francés y debe ajustar la última cuota cuando sea necesario para garantizar que la suma de amortizaciones sea exactamente igual al capital y que el saldo final sea Q0.00.

Estados del crédito

El ciclo contempla los estados SOLICITADO, APROBADO, DESEMBOLSADO, VIGENTE, EN_MORA, REESTRUCTURADO, CANCELADO, INCOBRABLE, además de los resultados RECHAZADO y ANULADO durante la creación del crédito. El modelo incluye deterioro y recuperación: VIGENTE puede pasar a EN_MORA y EN_MORA puede regresar a VIGENTE cuando el cliente regulariza todo lo vencido.

Mora 1, Mora 2, Mora 3 y Vencido no son estados del crédito; son tramos calculados según los días de atraso. Un crédito incobrable queda fuera de la cartera activa y un pago posterior se registra como recuperación de incobrable.

Actividad: origen del crédito

El crédito inicia con la solicitud del cliente. Se verifica o registra al cliente, se registra la solicitud, se recopila la información de evaluación y se validan las políticas. El Comité puede rechazar o aprobar. Una solicitud aprobada puede anularse antes del desembolso si el cliente desiste. Si continúa, se obtiene la política financiera vigente, se genera el plan, se registra el desembolso y el crédito queda vigente.

Matriz de trazabilidad

ID	Requisito	Caso de uso	Clase / módulo
R1	Registrar y consultar clientes	Registrar / Consultar cliente	Cliente · Origen
R2	Registrar solicitudes de crédito	Solicitar crédito	SolicitudCredito · Origen
R3	Evaluar solicitudes según política	Evaluar solicitud	SolicitudCredito · PoliticaCredito
R4	Aprobar o rechazar solicitudes	Aprobar / Rechazar solicitud	SolicitudCredito
R5	Desembolsar créditos aprobados	Desembolsar crédito	Credito · Movimiento
R6	Generar plan de amortización	Generar plan	PlanAmortizacion · Cuota
R7	Representar dinero sin errores de punto flotante	Casos con importes	Dinero
R8	Calcular interés corriente	Generar plan	PlanAmortizacion · Cuota
R9	Ajustar última cuota a saldo cero	Generar plan	PlanAmortizacion · Cuota
R10	Calcular días de atraso	Registrar pago / Clasificar mora	Credito · CalculadoraMora
R11	Calcular mora solo sobre capital	Registrar pago	CalculadoraMora
R12	Clasificar tramo de mora	Clasificar mora	CalculadoraMora · Credito
R13	Registrar pagos	Registrar pago	Pago · Credito
R14	Aplicar prelación de pagos	Registrar pago	PrelacionPago · Pago
R15	Permitir pagos parciales	Registrar pago	Pago · PrelacionPago · Cuota
R16	Manejar excedentes de pago	Registrar pago	Pago · PrelacionPago
R17	Evitar pagos duplicados	Registrar pago	Pago · Idempotencia
R18	Regularizar crédito en mora	Registrar pago	Credito · Estados
R19	Subir o bajar tramo de mora	Registrar pago / Clasificar mora	Credito · CalculadoraMora
R20	Reestructurar créditos autorizados	Reestructurar crédito	Credito
R21	Declarar incobrable >120 días	Declarar incobrable	Credito · Cartera

ID	Requisito	Caso de uso	Clase / módulo
R22	Registrar recuperaciones de incobrables	Registrar recuperación	Movimiento
R23	Consultar saldo	Consultar saldo	Credito · PlanAmortizacion
R24	Mantener movimientos auditables	Pagos / Desembolso / Cierres	Movimiento
R25	Generar cierre diario	Generar cierre diario	Cierre
R26	Generar cierre mensual	Generar cierre mensual	Cierre
R27	Garantizar cierre idempotente	Generar cierre	Cierre
R28	Calcular cartera activa	Consultar cartera en riesgo	CalculadoraCarteraRiesgo
R29	Calcular cartera en riesgo	Consultar cartera en riesgo	CalculadoraCarteraRiesgo · Credito
R30	Incluir reestructurados en riesgo	Consultar cartera en riesgo	CalculadoraCarteraRiesgo
R31	Excluir incobrables de cartera activa	Consultar cartera en riesgo	CalculadoraCarteraRiesgo
R32	Registrar historial de estados	Cambios de estado	HistorialEstadoCredito · Credito
R33	Usar políticas versionadas	Evaluar / Desembolsar / Mora	PoliticaCredito
R34	Consulta gerencial de cartera	Consultar cartera en riesgo	Cierre · CalculadoraCarteraRiesgo
R35	Aceptar distintos canales de pago	Registrar pago	Pago · Contratos/API

E2 - Decisión y justificación de arquitectura

Arquitectura seleccionada

Se selecciona un monolito modular basado en arquitectura hexagonal (Ports and Adapters). Esta decisión permite mantener las reglas financieras aisladas de la infraestructura y evita la complejidad operativa y de consistencia que implicaría introducir microservicios en esta etapa.

Atributos de calidad prioritarios

Prioridad	Atributo	Justificación
1	Fiabilidad y exactitud	Los cálculos financieros deben ser reproducibles y correctos a centavos.

Prioridad	Atributo	Justificación
2	Mantenibilidad	Las políticas, tasas y reglas pueden cambiar sin obligar a modificar todo el sistema.
3	Testabilidad	El núcleo debe poder probarse sin servidor, interfaz ni base de datos.
4	Auditabilidad y trazabilidad	Pagos, movimientos, estados y cierres deben poder reconstruirse y auditarse.
5	Modificabilidad y extensibilidad	La solución debe admitir posteriormente API, frontend, chat, RAG y MCP.

Organización modular

- **Origen:** Clientes, solicitudes, evaluación, aprobación y desembolso.
- **Cálculo financiero:** Dinero, amortización, intereses, mora y redondeos.
- **Cartera y Cobros:** Pagos, prelación, saldos y clasificación de mora.
- **Cierres:** Cierre diario, cierre mensual y cartera en riesgo.
- **Contratos / API:** Exposición de los casos de uso hacia adaptadores externos.

Arquitectura hexagonal

Los adaptadores primarios, como API REST, interfaz de usuario, chat o servidor MCP, invocan puertos primarios representados por los casos de uso. Estos coordinan el núcleo del dominio. Cuando el dominio necesita persistencia, reloj u otros servicios externos, depende de puertos secundarios definidos como interfaces. La infraestructura implementa dichos puertos. De esta forma, el dominio no conoce PostgreSQL, HTTP ni tecnologías de interfaz.

Modelo 4+1

Vista lógica

La vista lógica organiza el sistema en Origen, Cálculo financiero, Cartera y Cobros, Cierres y Contratos/API. Crédito se relaciona con PlanAmortizacion y Dinero; Pago utiliza PrelacionPago; los cálculos de mora se delegan a CalculadoraMora y los cierres utilizan CalculadoraCarteraRiesgo.

Vista de desarrollo

El código se organizará en tres áreas principales: dominio, aplicación e infraestructura. Dominio contiene las entidades y reglas puras; aplicación contiene casos de uso y puertos; infraestructura contiene persistencia, API y adaptadores técnicos. Las dependencias se orientan hacia el dominio.

Vista de escenarios

Los escenarios arquitectónicos principales son: aprobar y desembolsar un crédito, registrar un pago de cuota y generar un cierre. Los dos primeros ya se encuentran detallados mediante diagramas de secuencia de E1 y sirven para validar la colaboración entre actores, casos de uso, dominio y persistencia.

Modelo C4

Nivel 1 - Contexto

El Sistema de Gestión de Microcrédito interactúa con clientes, asesores, Comité de crédito, empleados de cobros y Gerencia. También se contempla un canal de pago externo. Para el Proyecto Final se prevén el asistente de chat y el servidor MCP como nuevos puntos de entrada hacia los mismos casos de uso.

Nivel 2 - Contenedores

La evolución prevista contempla un frontend React + Vite, un backend Express o Fastify, PostgreSQL como sistema de persistencia y, posteriormente, componentes de Chat/RAG y servidor MCP. Estos elementos se diseñan en E2 pero no se implementan todavía en el núcleo del Proyecto 1.

Nivel 3 - Componentes

Dentro del backend se distinguen adaptadores primarios, capa de aplicación, núcleo de dominio y puertos secundarios. API REST, Chat/RAG y MCP reutilizan casos de uso como DesembolsarCredito, RegistrarPago, GenerarCierre y ConsultarCarteraEnRiesgo. La persistencia se conecta mediante interfaces de repositorio, manteniendo el principio de inversión de dependencias.

Justificación arquitectónica

La arquitectura hexagonal implementada como monolito modular permite que el núcleo del sistema calcule planes de amortización, intereses, mora, aplicación de pagos y cartera en riesgo sin depender de bases de datos, interfaces gráficas o servicios externos. Esto favorece la exactitud, testabilidad, mantenibilidad y evolución de la solución.

Los futuros adaptadores, incluyendo API REST, frontend, asistente conversacional y servidor MCP, podrán consumir los mismos casos de uso. La decisión evita duplicar reglas de negocio y permite sustituir infraestructura sin modificar el dominio.

E3 - Diseño de componentes, principios y patrones

El diseño conserva las fronteras establecidas en E2 y lleva el módulo de cálculo financiero a nivel de clases e interfaces. Las reglas financieras permanecen en el dominio; los casos de uso coordinan el flujo y los adaptadores resuelven tecnologías externas.

Descomposición de componentes

Módulo	Componentes principales	Responsabilidad
Origen	Cliente, SolicitudCredito, PoliticaCredito, DesembolsarCredito	Registro, evaluación, aprobación, rechazo y desembolso.
Cálculo financiero	Dinero, PlanAmortizacion, AmortizacionFrancesa, CalculadoraMora	Cuotas, intereses, redondeo y exactitud monetaria.
Cartera y Cobros	Credito, Pago, PrelacionPago, ClasificadorMora	SalDOS, pagos, mora, estados y regularización.
Cierres	Cierre, Movimiento, CalculadoraCarteraRiesgo	Cortes diario/mensual, cartera activa y riesgo.
Contratos/API	Controladores, DTO, esquemas Zod, ErrorEnvelope	Exposición sin contener reglas de negocio.

Diseño detallado del cálculo financiero

Dinero encapsula monto y moneda, es inmutable y utiliza bigint para almacenar centavos. PlanAmortizacion coordina una estrategia de cálculo; AmortizacionFrancesa genera cuotas con decimal.js para tasas y operaciones intermedias y convierte cada resultado a Dinero mediante redondeo ROUND_HALF_UP. En la última cuota se amortiza todo el saldo restante para asegurar que la suma de amortizaciones sea exactamente el capital y que el saldo final sea Q0.00.

Aplicación de SOLID y GRASP

Principio	Aplicación en el proyecto
SRP / Alta cohesión	Cada clase concentra una regla: dinero, amortización, mora, pagos, cartera o estados.
OCP	Nuevas estrategias de amortización o políticas se agregan sin modificar el motor existente.
LSP	Las estrategias y estados se sustituyen respetando sus contratos.
ISP	Los puertos se definen por caso de uso y evitan interfaces generales innecesarias.
DIP	Dominio y aplicación dependen de abstracciones como repositorios y Reloj.
Experto en información	El cálculo vive en el objeto o servicio que posee los datos necesarios.
Controlador	Los casos de uso reciben solicitudes y coordinan objetos del dominio.

Principio	Aplicación en el proyecto
Polimorfismo	Strategy y State sustituyen decisiones condicionales extensas.

Patrones de diseño

Patrón	Ubicación	Justificación
Value Object	Dinero	Elimina errores de punto flotante, prohíbe mezclar monedas y conserva inmutabilidad.
Strategy (GoF)	Amortización y políticas financieras	Permite sustituir algoritmos y mantener políticas versionadas.
Chain of Responsibility (GoF)	Prelación de pagos	Cada eslabón consume gastos, mora, interés o capital y pasa el remanente.
State (GoF)	Ciclo de vida de Credito	Hace explícitas las transiciones válidas y rechaza operaciones incompatibles.
Specification	Tramos de mora	Representa reglas de clasificación componibles y verificables.
Repository	Puertos secundarios	Aísla persistencia y permite implementaciones en memoria durante pruebas.

Cohesión y acoplamiento

Los módulos presentan cohesión alta porque agrupan reglas que cambian por la misma causa. El acoplamiento entre módulos se limita a contratos y objetos del dominio. Contratos/API no calcula importes; Cálculo financiero no persiste; Cartera y Cobros no define tasas; y Cierres no modifica créditos. Esta separación facilita pruebas unitarias y evolución hacia adaptadores REST, chat y MCP.

E4 - Núcleo TypeScript ejecutable y pruebas

El walking skeleton implementa únicamente el núcleo de dominio en TypeScript estricto. No requiere servidor HTTP, base de datos, interfaz gráfica, autenticación, RAG ni MCP. La implementación demuestra que el diseño de E3 produce resultados financieros verificables.

Estructura implementada

Archivo	Responsabilidad	Patrón / criterio
dinero.ts	Montos en bigint, moneda, suma y resta inmutables.	Value Object
plan-amortizacion.ts	Método francés y ajuste de última cuota.	Strategy
calculadora-mora.ts	Interés moratorio sobre capital vencido.	Servicio de dominio
prelacion-pago.ts	Gastos → mora → interés corriente → capital.	Chain of Responsibility
cartera.ts	Tramos y cálculo de cartera en riesgo.	Specification
credito.ts	Estados, atraso y regularización.	State

Representación y redondeo

- Los importes monetarios se guardan como bigint en centavos dentro de Dinero.
- Las tasas, porcentajes y operaciones financieras intermedias utilizan decimal.js.
- Cada resultado monetario se redondea a dos decimales con ROUND_HALF_UP.
- La última cuota absorbe el saldo restante; no queda residuo de Q0.01.
- La mora se calcula únicamente sobre capital vencido, evitando anatocismo.

Evidencia de ejecución

Prueba obligatoria	Resultado validado
Ejecución general	6 archivos de prueba; 58 pruebas aprobadas.
Amortización	Q10,000.00, TNA 36%, 12 cuotas; cuotas 1-11 de Q1,004.62 y cuota 12 de Q1,004.63.
Invariantes	Suma de amortizaciones Q10,000.00 y saldo final Q0.00, ambos exactos.
Mora	$Q1,000.00 \times 26.5\% \times 10 / 365 = Q7.26$.
Cartera en riesgo	7.00%; luego 6.06% al declarar C-005 incobrable.
Reversibilidad	45 días → MORA_2; pago parcial a 10 días → MORA_1; regularización total → VIGENTE.
Transición inválida	El pago de un crédito SOLICITADO es rechazado por diseño.

Demostración de cálculos

El comando `npm run demo` compila el proyecto, muestra en consola el plan de amortización, la mora, la prelación, la cartera en riesgo y la reversibilidad, y genera `resultado-calculos.txt` como evidencia de ejecución. Este archivo es una salida y permanece excluido del repositorio mediante `.gitignore`.

E5 - Contratos de API y decisiones de arquitectura

La especificación OpenAPI 3.0.3 cubre los recursos principales y se encuentra en revisión final. Su base es consistente con la arquitectura hexagonal: los contratos exponen casos de uso y utilizan una convención uniforme de errores, sin trasladar reglas financieras a los controladores.

Recursos y operaciones

Recurso	Operaciones cubiertas	Consideraciones
Clientes	Registrar y consultar.	Validación uniforme de entrada.
Solicitudes	Crear, consultar, aprobar y rechazar.	El desembolso debe partir de una solicitud aprobada.
Créditos	Consultar crédito, saldo y plan.	Estados y tramos permanecen separados.
Pagos	Registrar pago.	Idempotency-Key obligatorio; evita aplicación duplicada.
Cierres	Generar cierre diario y mensual.	Reintentos retornan el cierre existente.
Cartera en riesgo	Consultar capital activo y en riesgo.	Incluye reestructurados; excluye incobrables activos.

Convención de errores e idempotencia

Las respuestas de error utilizan un envelope uniforme con código, mensaje y detalles, junto con códigos HTTP coherentes. Para pagos, una operación nueva devuelve 201; un reintento idéntico con la misma clave devuelve 200 sin aplicar nuevamente el pago; reutilizar la clave con un cuerpo diferente devuelve 409. Los cierres diario y mensual también se diseñan como operaciones idempotentes por fecha de corte.

ADR-001 - Arquitectura hexagonal con monolito modular

Campo	Síntesis
Estado / fecha	Aceptada · 27/08/2026
Contexto	El dominio financiero necesita exactitud, auditabilidad y múltiples canales futuros sin duplicar reglas.
Decisión	Dominio independiente; puertos para casos de uso y servicios externos; adaptadores dependientes de los puertos; un solo desplegable modular.
Consecuencias	Pruebas sin infraestructura, sustitución de tecnologías y reutilización del dominio; costo de abstracciones y disciplina de límites.

ADR-002 - Representación de importes monetarios

Campo	Síntesis corregida
Estado / fecha	Aceptada · 27/08/2026; documento pendiente de actualización formal.
Contexto	Los importes exigen precisión exacta y el punto flotante no es aceptable.
Decisión	Dinero inmutable con bigint en la unidad mínima; decimal.js para tasas y cálculos intermedios; conversión final con ROUND_HALF_UP.
Consecuencias	Exactitud e invariantes verificables; conversiones explícitas y disciplina para monedas, tasas y redondeo.

Conclusiones y verificación final

Los entregables E1 a E5 describen una misma solución: el modelo UML establece las reglas y entidades; E2 separa dominio e infraestructura; E3 asigna responsabilidades y patrones; E4 valida el diseño con código y 58 pruebas; y E5 expone los casos de uso mediante contratos consistentes. La arquitectura elegida permite agregar posteriormente REST, interfaz, chat y MCP como adaptadores, sin duplicar el cálculo financiero.

Glosario

Término	Definición
ADR	Registro de una decisión arquitectónica, su contexto y consecuencias.
C4	Modelo que representa contexto, contenedores, componentes y código.
Cartera en riesgo	Saldo de capital con más de 30 días de atraso y créditos reestructurados; incobrables se excluyen de cartera activa.
Dinero	Objeto de valor inmutable que representa monto en unidad mínima y moneda.
Idempotencia	Repetir la misma operación no duplica su efecto.
Puerto / adaptador	Contrato del núcleo y componente que conecta una tecnología externa.
Tramo de mora	Clasificación derivada de días de atraso; no equivale al estado INCOBRABLE.
Walking skeleton	Núcleo ejecutable mínimo que valida el diseño con pruebas.

Control de cambios

Versión	Fecha	Responsable	Descripción
1.0	28/08/2026	Pendiente	Primera versión de consolidado de E1-E5.